

Sistemas Operacionais 2

OVERVIEW DE PROCESSOS NO MINIX

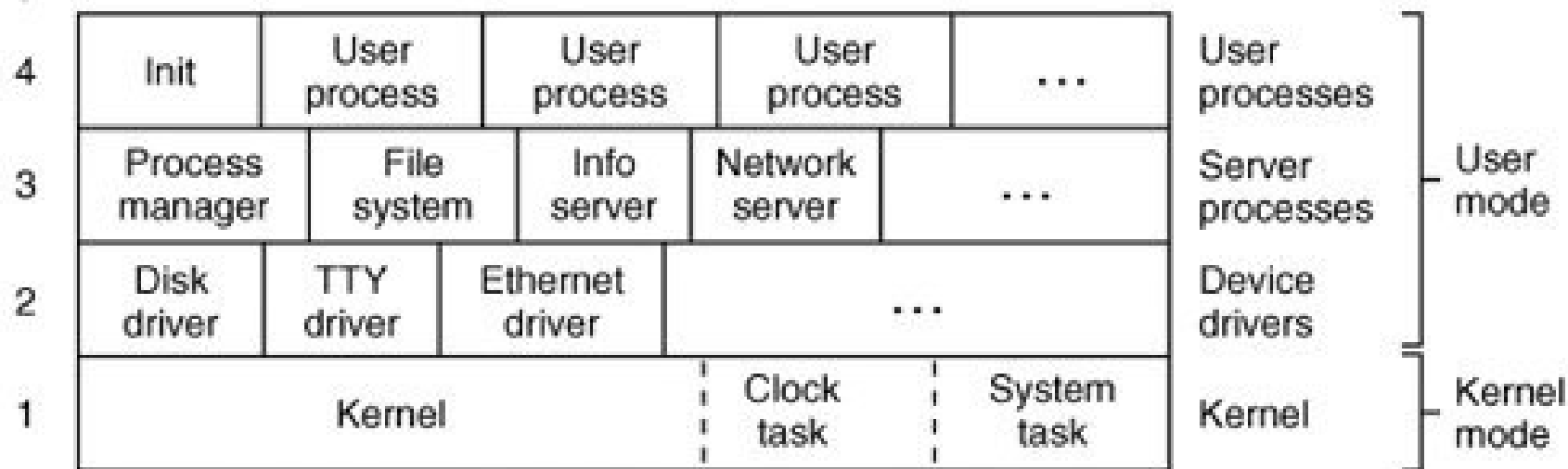
Professor.: Dalton Matsuo Tavares

Nota: Alguns slides e/ou figuras nesta apresentação foram adaptadas de slides de TANENBAUM e WOODHULL ©2006. Cortesia de Dalton Matsuo Tavares.

- UNIX → kernel monolítico
- MINIX 3 → coleção de processos, que se comunicam entre si e com processos de usuário.
- Usa uma única primitiva de comunicação por passagem de mensagens.
 - Torna simples, por exemplo, substituir o sistema de arquivos inteiro, sem a necessidade de recompilar o kernel.

ESTRUTURAÇÃO DO MINIX

Layer



- Kernel
 - Responsável pela transição entre estados (pronto, executando, bloqueado).
 - Lida com passagem de mensagens entre processos.
 - Acesso a portas de I/O e interrupções.
- Tarefa de *clock*
 - Dispositivo de I/O que interage com o hardware que gera sinais de temporização.
 - Não é acessível pelo usuário, como no caso de outros dispositivos.

- Tarefa de sistema
 - Implementa as chamadas de sistema
- Grande parte do kernel é escrita em C.
 - Manipulação de interrupções, chaveamento de contexto, gerenciamento de MMU são implementados em assembly → porções diretamente relacionadas ao hardware.

- As outras 3 camadas poderiam ser tratadas como se fossem uma única camada.
 - Escalonadas para execução sobre o kernel.
 - Não podem acessar portas de I/O diretamente.
 - Não podem acessar segmentos de memória que não tenham sido alocados a elas.
 - Diferenciação entre camadas diz respeito aos privilégios que cada uma possui para executar chamadas no kernel.
 - Camadas 2, 3 e 4 possuem privilégios decrescentes nesta ordem.

- **Camada 2** hospeda os controladores de dispositivos.
 - Podem realizar chamadas de kernel.
 - Ex. podem solicitar a um dispositivo, por intermédio de uma chamada de kernel, a leitura de I/O.

- Camada 3 contém servidores.
 - Oferecem serviços úteis aos processos de usuário.
 - *Process manager* (PM): início/término (ex. *fork*, *exec*, *exit*), sinais relacionados a execução de processos (ex. *alarm*, *kill*). Gerenciamento de memória (ex. *brk*).
 - *File System* (FS): chamada de sistemas para o sistema de arquivos (ex. *read*, *mount*, *chdir*).
 - *Information Server* (IS): informação de *debug* e *status* sobre outros servidores. Ótimo para desenvolvimento.

- *Reincarnation Server* (RS): inicia e se necessário, reinicia *drivers* de dispositivos que não são carregados na memória no mesmo momento do kernel. Contribui para a tolerância a falha do sistema.
- *Network Server* (*inet*): responsável pela transferência de dados em rede.

Modo de Usuário (cont.)

- Chamadas de kernel $\neq$ chamadas de sistema POSIX
 - Chamadas de kernel são funções de baixo nível fornecidas pela tarefa de sistema que permitem que *drivers* e servidores façam seu trabalho. ex. Leitura de porta de I/O.
 - Chamadas POSIX (ex. *read*, *fork*, *unlink*), são chamadas de alto nível, definidas pelo padrão POSIX e estão disponíveis para programas de usuário no nível 4.
 - Chamadas de kernel podem ser considerados um subconjunto especial de chamadas de sistema.

- **Camada 4**: shells, editores, compiladores, e programas escritos por usuários (a.out).
- **NOTA**: no MINIX 3, *device drivers* foram implementados completamente em espaço de usuário. As únicas exceções são a tarefa de *clock* e a tarefa de sistema.
 - Logo: *device driver* $\neq$ tarefa (*task*)

GERENCIAMENTO DE PROCESSO NO MINIX

- Segue o modelo geral de processos tradicional.
- Hierarquia de processos (pais e filhos).
- Todos os processos de usuário são parte de uma única árvore com o init como raiz.
- Servidores e *drivers* são um caso especial – devem ser iniciados antes de qualquer processo de usuário – incluindo o init.

- Hierarquia de disco de *boot* (configurável)
 - *Floppy* → *CD-ROM* → 1.^o disco rígido
- Procura o programa de *bootstrap* → tipicamente um programa pequeno (512 bytes) para caber em um setor do disco.
- Posteriormente carrega um programa maior (o *boot*) que carrega o sistema operacional.

STARTUP (cont.)

- Discos rígidos precisam de um passo adicional
 - Tipicamente dividido em partições.
 - Primeiro setor contém um pequeno programa e a tabela de partição do disco.
 - Chamado de *master boot record*.
 - O programa é executado para ler a tabela de partição e selecionar a partição ativa.
 - A partição ativa possui um *bootstrap* no primeiro setor, o qual é carregado e executado para encontrar uma cópia do *boot* na partição.

- CD-ROMs
 - Pode carregar um grande bloco de dados na memória de imediato.
 - Carrega tipicamente a cópia exata de um disco de boot (ex. *floppy*), a qual é inserida na memória (RAM *disc*). Posteriormente o controle é passado ao RAM disc e o procedimento é análogo a um *floppy*.

STARTUP (cont.)

- O programa *boot* procura um arquivo multi-parte específico no *floppy* ou partição e carrega as partes individuais na memória nos locais apropriados → imagem de *boot*.
 - Partes mais importantes incluem o *kernel*, *process manager* (PM) e *file system* (FS).
 - PM + FS → ***reincarnation server***, RAM disk, *console*, *log drivers* e *init*.

STARTUP (cont.)

Component	Description	Loaded by
kernel	Kernel + clock and system tasks	(in boot image)
pm	Process manager	(in boot image)
fs	File system	(in boot image)
rs	(Re)starts servers and drivers	(in boot image)
memory	RAM disk driver	(in boot image)
log	Buffers log output	(in boot image)
tty	Console and keyboard driver	(in boot image)
driver	Disk (at, bios, or floppy) driver	(in boot image)
init	parent of all user processes	(in boot image)
floppy	Floppy driver (if booted from hard disk)	/etc/rc
is	Information server (for debug dumps)	/etc/rc
cmos	Reads CMOS clock to set time	/etc/rc
random	Random number generator	/etc/rc
printer	Printer driver	/etc/rc

- Particularidades do MINIX

- Init é o primeiro processo de usuário → último processo carregado como parte da imagem de boot.
- Já existem alguns processos de sistema quando o init é executado.
 - ex. *clock task* e *system task* (sem PIDs, não são considerados parte da árvore de processos).
 - PM → primeiro processo a ser executado em espaço de usuário – não é filho ou pai de qualquer outro processo.
 - RS é o pai de todos os outros processos iniciados a partir da imagem de *boot* (ex. *drivers* e servidores) → deve ser informado se algum destes precisar ser reiniciado.

INICIALIZAÇÃO DA ÁRVORE DE PROCESSOS (cont.)

- O PID 1 é reservado ao init.
- É um dos filhos do RS.
- Executa primeiro o shell script /etc/rc e dispara drivers e servidores adicionais que não são parte da imagem de boot.
 - Programas iniciados pelo rc são filhos do init.
 - Inicia uma ferramenta chamada service.
 - Interface de usuário para o RS. O RS inicia um programa qualquer e o converte para um processo de sistema.
 - RS adota todos os processos de sistema exceto o PM. ex. *floppy*, *cmos* e IS (*information Server*).

INICIALIZAÇÃO DA ÁRVORE DE PROCESSOS (cont.)

- Após a inicialização do cmos o script rc pode iniciar o *real time clock*.
- Até aqui, todos os arquivos necessários encontrados no dispositivo root (/bin e /sbin).
- Após o *startup*, outros *filesystems* são montados (ex. /usr/adm/wtmp)
- Init lê arquivo /etc/ttytab → lista os dispositivos terminais potenciais

INICIALIZAÇÃO DA ÁRVORE DE PROCESSOS (cont.)

- O usuário digita seu nome no prompt de login → `/usr/bin/login` → verificação de senha → shell de usuário (`/bin/sh` ou outro).
- **CONCLUSÃO:** exceto tarefas compiladas no kernel e o PM, todos os processos, de sistema e de usuário, formam uma árvore.
 - Diferente da árvore de processo de um sistema UNIX convencional, o `init` não está na raiz da árvore, e a estrutura da árvore não permite que se determine a ordem na qual os processos de sistema foram iniciados.

- As principais chamadas de sistema para gerenciamento de processos no MINIX 3 são fork e exec.
 - fork → cria um processo.
 - exec → permite que um processo execute um determinado programa.
 - Informação sobre o processo mantida na tabela de processos.
 - Dividida entre o kernel, PM e FS. Após criação/terminação de processos, o PM atualiza sua parte na tabela de processos e notifica o kernel e o FS para que façam o mesmo.

- 3 primitivas são oferecidas para envio e recebimento de mensagens. São chamadas pelos procedimentos de biblioteca do C.
 - `send(dest, &message);` → para enviar uma mensagem ao processo `dest`,
 - `receive(source, &message);` → para receber uma mensagem do processo `source` (ou `ANY`), e
 - `sendrec(src_dst, &message);` → para enviar uma mensagem e esperar por uma resposta do mesmo processo.

- O segundo parâmetro em cada chamada é o endereço local dos dados da mensagem.
- O mecanismo de passagem de mensagem no kernel copia a mensagem do emissor para o receptor.
- A resposta (para o sendrec) sobrescreve a mensagem original.

- Cada tarefa, *driver* ou processo servidor pode trocar mensagens somente com alguns outros processos.
 - Esta política é reforçada pela representação em camadas apresentada.
- Processos de usuário não podem enviar mensagens entre si.

- Quando um processo envia uma mensagem a um processo que não está esperando, o emissor bloqueia até que o destino realize um receive.
- MINIX usa o método de *rendezvous*.
- **Vantagem**: abordagem simples e evitar problemas de buffer de mensagens enviadas, mas ainda não recebidas. (i.e. superar a capacidade do *buffer*).
- Mensagens de tamanho fixo → previne estruturalmente erros de buffer overrun.

- Evita ocorrência de *deadlock* ($A \rightarrow B$ (block A), $B \rightarrow A$ (block B) $\Rightarrow$ *deadlock*).
- Algumas vezes, algo diferente de uma mensagem bloqueante é necessário.
 - Primitiva `notify(dest);` $\rightarrow$ emissor continua executando mesmo que o receptor não esteja esperando.
 - Informação limitada – geralmente um bit + informação de *timestamp*.
 - Usado entre processos de sistema.

COMUNICAÇÃO INTERPROCESSOS MINIX (cont.)

- Cada processo de sistema possui um *bitmap* para notificações pendentes.
- ex. $A \rightarrow B$, quando B finalmente realiza o *receive*, ele primeiro verifica o *bitmap* de notificações para verificar os *sends* que foram feitos a ele.

- Sistema de interrupção
 - Mantém um sistema operacional multiprogramado funcionando.
 - Processos bloqueiam quando eles fazem requisições por entrada
 - **permite que outros processos executem.**
 - Quando a entrada se torna disponível, o processo em execução é interrompido pelo disco, teclado etc.

ESCALONAMENTO DE PROCESSO NO MINIX (cont.)

- O *clock* gera interrupções
 - São usadas para garantir que um processo de usuário em execução, que não tenha solicitado entrada, eventualmente disponibilize a CPU.
 - É função da camada mais baixa do MINIX esconder estas interrupções pela transformação das mesmas em mensagens.
 - Quando um dispositivo de I/O completa sua operação, ele envia uma mensagem para algum processo, acionando-o e o tornando apto a executar.

ESCALONAMENTO DE PROCESSO NO MINIX (cont.)

- *Traps* - interrupções geradas por software.
 - Operações *send* e *receive* descritas anteriormente.
 - São traduzidas pela biblioteca de sistema em interrupções de software
 - Possuem exatamente o mesmo efeito de uma interrupção gerada por hardware.
- O processo que executa uma interrupção de software é imediatamente bloqueado e o kernel é ativado para processar a interrupção.

ESCALONAMENTO DE PROCESSO NO MINIX (cont.)

- Programas de usuário não se referem ao *send* ou *receive* diretamente.
 - A qualquer momento, uma das chamadas de sistema é invocada, direta ou por uma rotina de biblioteca.
 - **sendrec** é usado internamente e uma interrupção de software é gerada.

ESCALONAMENTO DE PROCESSO NO MINIX (cont.)

- Escalonamento: permite determinar qual processo merece a oportunidade de executar. Ocorre toda vez que um processo é interrompido (ex. I/O ou *clock*), ou devido a execução de interrupção de software (*traps*).
 - Isto deve ser feito sempre que um processo termina
 - No MINIX, interrupções devido a operações de I/O, ou *clock* ou passagem de mensagem ocorrem mais freqüentemente que terminação de processo.

ESCALONAMENTO DE PROCESSO NO MINIX (cont.)

- O escalonador do MINIX usa um sistema de fila multinível.
 - 16 filas são definidas
 - É possível recompilar o kernel para usar mais ou menos filas.
 - A prioridade mais baixa é usada somente para processos ociosos (*IDLE*) os quais executam quando não existe nada a fazer.
 - Processos de usuário começam por padrão em uma fila vários níveis acima do nível mais baixo.

ESCALONAMENTO DE PROCESSO NO MINIX (cont.)

- Servidores são normalmente escalonados em filas com prioridades mais altas do que o permitido para um processo de usuário.
- **Drivers** são escalonados em filas com prioridade maior do que servidores.
- **Clock e tarefas de sistema** são escalonados na fila de prioridade mais alta.
- **Nem todas as 16 filas disponíveis estarão em uso em um dado momento.**
 - Esses níveis extra estão disponíveis para experimentação

ESCALONAMENTO DE PROCESSO NO MINIX (cont.)

- Processos são iniciados em apenas algumas filas.
- Um processo pode ser movido para uma fila de prioridade diferente pelo sistema
 - Como drivers adicionais são adicionados ao MINIX, os ajustes padrão podem ser configurados para melhor desempenho.
 - Dentro de certos limites, o usuário pode usar o comando **nice** para alterar prioridades para um dado utilitário.
 - Explore-o usando as páginas de manual.

ESCALONAMENTO DE PROCESSO NO MINIX (cont.)

- Ex. caso se deseje adicionar um servidor para envio de *stream* de audio ou video digital pela rede, pode ser atribuído a tal servidor uma prioridade de início mais alta do que a de servidores atuais.
- A prioridade inicial de um servidor corrente ou *driver* pode ser reduzida para que o novo servidor alcance melhor desempenho.

ESCALONAMENTO DE PROCESSO NO MINIX (cont.)

- ***Quantum***: intervalo de tempo permitido antes que um processo seja interrompido (*preempted*).
 - Não é o mesmo para todos os processos.
 - Processos de usuário possuem um quantum relativamente baixo.
 - *Drivers* e servidores devem executar somente até que bloqueiem. São preemptáveis, mas possuem um *quantum* maior.
 - Podem executar por um número finito de *ticks* de relógio.

ESCALONAMENTO DE PROCESSO NO MINIX (cont.)

- Se usarem seu *quantum* inteiro, são preemptados para não travar o sistema.
- O processo que sofreu *timeout* será considerado pronto, e pode ser colocado no final da fila.
- **Se um processo que tenha usado seu quantum inteiro executa novamente e em sequência**, pode ser que este esteja preso em um laço evitando que outros processos com prioridade mais baixa executem.
 - **Solução:** abaixar a prioridade do processo, colocando-o em uma fila de prioridade mais baixa.
 - Se o processo chegar a um *timeout* novamente e outro processo ainda não puder executar, sua prioridade será abaixada novamente.
 - Eventualmente, algum outro processo deve ter a chance de executar.

ESCALONAMENTO DE PROCESSO NO MINIX (cont.)

• Um processo que teve sua prioridade abaixada pode recuperá-la e voltar a uma fila de maior prioridade.

- Se um processo usa todo o seu *quantum*, mas não evita que outros processos executem, ele será promovido para uma fila de prioridade mais alta, até uma prioridade máxima permitida.
 - Tal processo aparentemente precisa de seu *quantum*, mas não prejudica os demais processos.

ESCALONAMENTO DE PROCESSO NO MINIX (cont.)

- De outra forma, processos são escalonados usando uma versão levemente modificada de **round robin**.
 - Se um **processo** usa parte do seu *quantum* e se torna indisponível, interpreta-se que ele bloqueou **esperando por I/O → quando ele se torna pronto novamente ele é colocado na cabeça da fila, mas somente com a parte restante de seu *quantum*.**
 - Pretende-se dar ao processo de usuário uma resposta rápida para I/O.
 - **Um processo que se tornou indisponível porque usou seu *quantum* inteiro é colocado no fim da fila usando um algoritmo *round robin* padrão.**

ESCALONAMENTO DE PROCESSO NO MINIX (cont.)

- A escala de prioridade:
 - (+) Tarefas (*tasks*) → *drivers* → servidores → processos de usuário (-)
- Ao selecionar um processo para executar, o escalonador verifica se existem processos na fila de mais alta prioridade.
 - Se um ou mais processos estão prontos, aquele na cabeça da fila irá executar.

ESCALONAMENTO DE PROCESSO NO MINIX (cont.)

- Caso nenhum processo esteja pronto, o próximo processo de prioridade mais baixa será testado, e assim por diante.
- Como *drivers* respondem a requisições de servidores, e servidores respondem a requisições de processos de usuário, eventualmente, todos os processos de prioridade mais alta devem completar seja qual for o trabalho solicitado a eles.
 - Drivers essenciais e servidores recebem um quantum tão grande que normalmente eles nunca são preemptados pelo clock.

ESCALONAMENTO DE PROCESSO NO MINIX (cont.)

- Se nenhum processo está pronto, o processo IDLE é escolhido.
 - Isto coloca a CPU em um modo de baixo consumo até que a próxima interrupção ocorra.
- A cada *tick* de *clock*, uma verificação é feita para ver se o processo corrente foi executado por mais do que um *quantum*.
 - Em caso positivo, o escalonador o move para o final da fila (pode não realizar nada se este está sozinho na fila).

Escalonamento de Processo no MINIX (cont.)

- Processo irá executar novamente de imediato
 - se não existe processos em filas de prioridade mais alta e se o processo anterior está sozinho em sua fila.
 - De outra forma, o processo na cabeça da fila de prioridade mais alta não-vazia será o próximo a executar.

ESCALONAMENTO DE PROCESSO NO MINIX (cont.)

- Caso algo dê errado, a prioridade deles pode ser temporariamente abaixada para prevenir que o sistema pare totalmente.
- Provavelmente, nada de útil pode ser feito se isto acontecer a um servidor essencial.
 - Pode ser possível desligar o sistema graciosamente, prevenindo perdas de dados e possivelmente reunindo informação que possa ajudar na depuração do problema.

- Convenções:

- Os termos "procedimento", "função", e "rotina" serão usados de forma intercambiável.
- Nomes de variáveis, procedimentos e arquivos serão escritos em itálico, como em *rw_flag*.
- As tarefas que são compiladas no kernel são identificadas por letras maiúsculas como em **CLOCK**, **SYSTEM** e **IDLE**.
- Chamadas de sistema serão representadas em minúsculo, courier new. Ex. `read`.
- Podem existir pequenas discrepâncias entre o código apresentado aqui e o código do MINIX.

- A implementação do MINIX é prevista para computadores de 32 bits (ex. 80386, 80486, Pentium, Pentium Pro, II, III, 4, M, ou D etc).
- O path para o código fonte C completo é `/usr/src`
- Um diretório importante é o `/usr/src/include` onde a cópia mestre dos arquivos de cabeçalho C estão localizadas.

- Cada diretório na árvore de diretórios contém um Makefile, o qual coordena a operação do utilitário make do UNIX.
- O Makefile controla a compilação dos arquivos em seu diretório.
 - Também pode coordenar a compilação de arquivos em um ou mais subdiretórios.
 - make assegura que todos os arquivos são compilados.

Organização do Código Fonte do MINIX (cont.)

- Testa módulos previamente compilados para ver se eles estão atualizados e recompila aqueles que foram modificados desde a última compilação.
- Evita a compilação de arquivos que não precisam ser recompilados.
- make direciona a combinação de módulos compilados separadamente em um programa executável e também pode gerenciar a instalação do programa completo.

- O todo ou parte da árvore `/usr/src` pode ser realocada, considerando que o Makefile em cada diretório de `src` usa um caminho relativo para os subdiretórios.
 - Cópia de segurança dos fontes de `/usr/src` para algum outro lugar.
 - Geração de patches.

Organização do Código Fonte do MINIX (cont.)

- Arquivos de cabeçalho C são um caso especial.
 - Os arquivos em `/usr/include` não são a “cópia mestre”.
 - Caso se deseje realizar modificações permanentes, estas devem ser realizadas em `/usr/src/include` (sempre que se compilar o sistema sobrescreve a cópia em `/usr/include`)
 - **Lembre-se:** em linguagem C os arquivos de cabeçalhos são inseridos desta forma:
 - `#include <arquivo>`
 - Inclui arquivos a partir `/usr/include/`.
 - `#include "arquivo"`
 - lê um arquivo no sistema de arquivo local (`.`).

- O diretório `/usr/include` possui alguns arquivos de cabeçalho padrão POSIX. Além disso, possui 3 subdiretórios:
 - `/usr/include/sys/`: arquivos de cabeçalho POSIX adicionais.
 - `/usr/include/minix/`: arquivos de cabeçalho usados pelo sistema operacional MINIX 3.
 - `/usr/include/ibm/`: arquivos de cabeçalho com definições específicas de IBM PC.

- Existem extensões ao MINIX 3.
 - Ex. /usr/include/arpa e /usr/include/net e seu subdiretório /usr/include/net/gen, suportam extensões de rede.
 - Não são necessários para compilar o sistema MINIX básico.

Organização do Código Fonte do MINIX (cont.)

- Além do `/usr/src/include`, o diretório `/usr/src/` contém três outros importantes subdiretórios com código fonte de sistema operacional
 - `/usr/src/kernel/`: camada 1 (escalonamento, mensagens, tarefas de clock e sistema)
 - `/usr/src/drivers/`: camada 2 (*device drivers* para disco, console impressora, etc.).
 - `/usr/src/servers/`: camada 3 (gerenciador de processo, sistema de arquivo, outros servidores)



• Outros 3 diretórios de código fonte não são discutidos, mas são essenciais para produzir um sistema funcional:

- `/usr/src/lib/`: código fonte para as bibliotecas (e.g. open, read etc).
- `/usr/src/tools/`: Makefile e scripts para construir o sistema MINIX 3.
- `/usr/src/boot/`: o código para iniciar e instalar o MINIX 3.

Organização do Código Fonte do MINIX (cont.)

- Códigos fonte importantes:
 - `/usr/src/servers/`: contém código fonte para o programa init e o servidor de reencarnação
 - `/usr/src/servers/inet/`: código fonte para o servidor de rede.
 - `/usr/src/drivers`: código fonte para device drivers (ex. drivers de disco, placas de som, adaptadores de rede).
 - `/usr/src/test/`: diretório com programas projetado para testar um sistema MINIX 3 recentemente compilado.
 - `/usr/src/commands`: com código fonte para os programas utilitários (ex. cat, cp, date, ls, pwd etc).

Compilando e Executando o MINIX

- Para compilar o MINIX 3, execute o make em `/usr/src/tools`.
 - Existem várias opções. Execute o make sem parâmetros.
 - Sugestão: `make image` (não instala)/`make hdimage` (instala).
 - `make image` gera uma nova cópia dos arquivos de cabeçalho em `/usr/src/include` e os copia para `/usr/include`.

Compilando e Executando o MINIX (cont.)

- Código fonte em `/usr/src/kernel` e vários subdiretórios de `/usr/src/servers` são compilados em arquivos objeto.
 - Todos os arquivos objeto em `/usr/src/kernel` são ligados (*linked*) para formar um único programa executável, o kernel.
 - Os arquivos objeto em `/usr/src/servers/pm` também são ligados juntos para formar um único programa executável, o pm.
 - Todos os arquivos objeto em `/usr/src/servers/fs` são ligados para formar o fs.

Compilando e Executando o MINIX (cont.)

- Os programas adicionais listados como parte da imagem de boot também são compilados e ligados em seus próprios diretórios.
 - Ex. rs e init em diretórios de /usr/src/servers e /usr/src/drivers/memory, /usr/src/drivers/log, /usr/src/drivers/tty.

Compilando e Executando o MINIX (cont.)

Component	Description	Loaded by
kernel	Kernel + clock and system tasks	(in boot image)
pm	Process manager	(in boot image)
fs	File system	(in boot image)
rs	(Re)starts servers and drivers	(in boot image)
memory	RAM disk driver	(in boot image)
log	Buffers log output	(in boot image)
tty	Console and keyboard driver	(in boot image)
driver	Disk (at, bios, or floppy) driver	(in boot image)
init	parent of all user processes	(in boot image)
floppy	Floppy driver (if booted from hard disk)	/etc/rc
is	Information server (for debug dumps)	/etc/rc
cmos	Reads CMOS clock to set time	/etc/rc
random	Random number generator	/etc/rc
printer	Printer driver	/etc/rc

Compilando e Executando o MINIX (cont.)

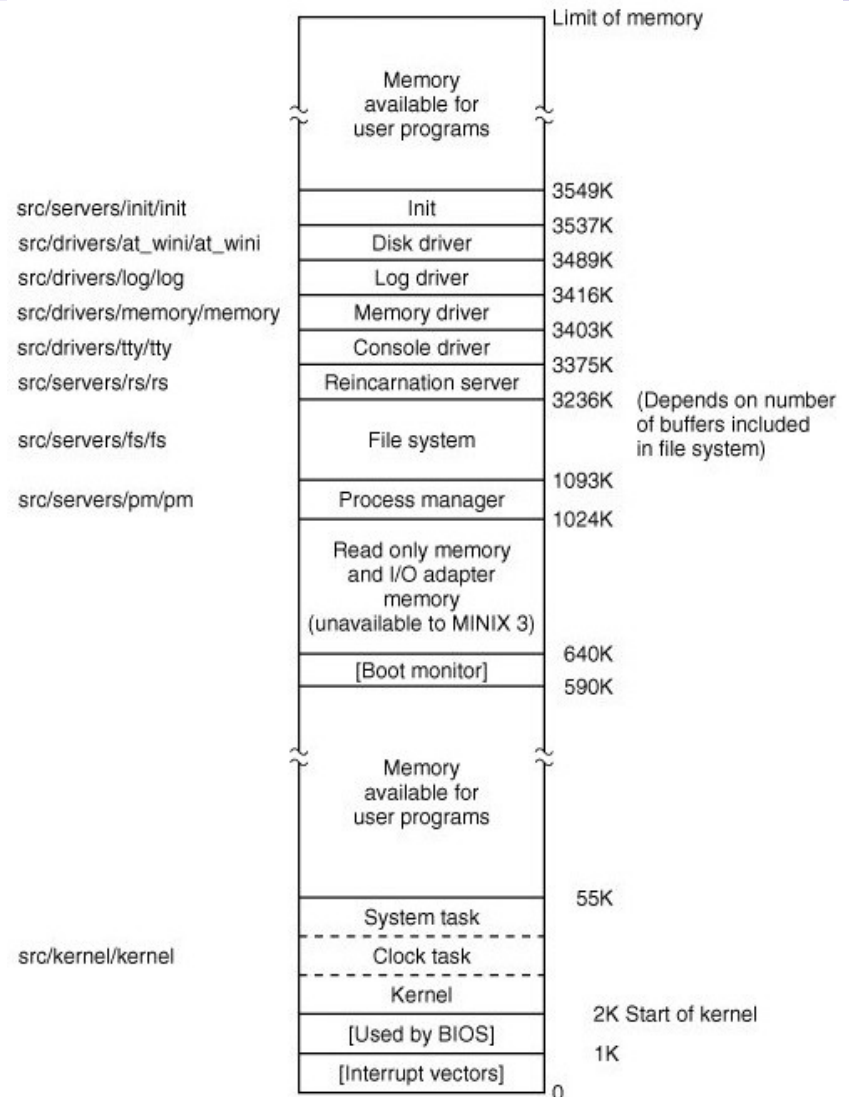
- Para instalar uma cópia de trabalho do MINIX 3 capaz de realizar boot, usa-se um programa chamado installboot (fonte em /usr/src/boot)
 - Adiciona nomes ao kernel, pm, fs, init e outros componentes da imagem de boot.
 - Ajusta (*padding*) esses componentes de modo que seu tamanho seja um múltiplo do tamanho do setor do disco (torna simples carregar partes independentemente), **e concatená-las em um único arquivo.**
 - Imagem de boot e pode ser copiado no diretório de boot ou em /boot/image de um disquete ou uma partição de um disco rígido.

Compilando e Executando o MINIX (cont.)

- Posteriormente, o boot monitor pode carregar a imagem de boot e transferir o controle para o sistema operacional.

Compilando e Executando o MINIX (cont.)

- Exemplo de **layout de memória** após os programas concatenados serem separados e carregados.



Compilando e Executando o MINIX (cont.)

- **Layout de memória:**

- O kernel é carregado em memória baixa, todas as outras partes da imagem de boot são carregadas acima de 1 MB.
- Quando programas de usuário são executados, a memória disponível acima do kernel será usada primeiro.
- Caso um novo programa não caiba lá, ele será carregado no escopo de memória alto, acima do init.
 - Detalhes dependem da configuração do sistema.
 - No exemplo, considera-se um MINIX 3 com sistema de arquivos configurado com um tamanho de bloco para o cache que possa manter **512 blocos de 4-KB**.

Compilando e Executando o MINIX (cont.)

- Quantidade modesta; recomenda-se mais caso se disponha de memória.
- Se o tamanho do bloco de cache for reduzido drasticamente, seria possível colocar o sistema inteiro em menos de 640 K de memória, com espaço para alguns processos de usuário.

Arquivos de Cabeçalho Tradicionais

- O diretório `/usr/include` e seus subdiretórios contém uma coleção de arquivos definindo constantes, macros e tipos.
- O padrão POSIX requer muitas destas definições e especifica em quais arquivos do diretório `/usr/include` e seu subdiretório `/usr/include/sys` elas se localizam.
 - Os arquivos nestes diretórios são cabeçalhos ou arquivos de include identificados pelo sufixo `.h`, e usados por meio de declarações do tipo `#include` em arquivos fonte C.

Arquivos de Cabeçalho Tradicionais (cont.)

- `/usr/src/include` → cabeçalhos tradicionalmente usados em programas de usuário.
- `/usr/src/include/sys/` → usado tradicionalmente para compilar programas de sistema e utilitários.
- Distinção não é de suma importância.

Arquivos de Cabeçalho Tradicionais (cont.)

- Os cabeçalhos de propósito mais geral não são diretamente referenciados por nenhum arquivo fonte C no MINIX 3.
 - Incluídos em outros arquivos de cabeçalho.
 - Cada grande componente do MINIX 3 possui um arquivo de cabeçalho mestre. Ex1.
/usr/src/kernel/kernel.h,
/usr/src/servers/pm/pm.h e
/usr/src/servers/fs/fs.h.

Arquivos de Cabeçalho Tradicionais (cont.)

- São incluídos em cada compilação de componentes.
- Ex2. Código fonte para cada um destes device drivers incluem um arquivo de certa forma similar, `/usr/src/drivers/drivers.h`
- Cada arquivo mestre é confeccionado para atender as necessidades da parte correspondente do MINIX 3, mas cada um começa com uma seção, a qual inclui a maioria dos arquivos mostrados aqui.
- Cabeçalhos de vários diretórios são usados juntos.

Arquivos de Cabeçalho Tradicionais (cont.)

```
#include <minix/config.h>      /* MUST be first */
#include <ansi.h>               /* MUST be second */
#include <limits.h>
#include <errno.h>
#include <sys/types.h>
#include <minix/const.h>
#include <minix/type.h>
#include <minix/syslib.h>
#include "const.h"
```

Parte do arquivo de cabeçalho mestre. Assegura a inclusão de arquivos de cabeçalho necessários por todos os fontes do C. **Nota:** dois arquivos const.h, um do /usr/include e outro do diretório local são referenciados.

Arquivos de Cabeçalho Tradicionais (cont.)

- **ansi.h** → teste se o compilador atende a requisitos do C padrão, conforme definido pela ISO. Definido por meio de macros.
- **limits.h** → define tamanhos básicos, como o n.o de bits em um inteiro, assim como limites no SO como o tamanho de um nome de arquivo.
- **errno.h** → contém os identificadores de erros que são retornados a programas de usuário na variável global `errno`.
 - Falha de chamada de sistema.
 - Erros internos – envio de mensagem a uma tarefa inexistente.

Arquivos de Cabeçalho Tradicionais (cont.)

- **unistd.h** → define constantes (POSIX).
 - Inclui protótipos para funções C, incluindo aquelas usadas para cessar chamadas de sistema MINIX.
- **string.h** → fornece protótipos para muitas funções C usadas para manipulação de string.
- **signal.h** → define os nomes de sinal padrão. Também contém protótipos para algumas funções relacionadas a processamento de sinais.

Arquivos de Cabeçalho Tradicionais (cont.)

- **fcntl.h** → define simbolicamente muitos parâmetros usados em operações de controle de arquivo.
- **termios.h** → define constantes, macros e protótipos de funções usados para controlar dispositivos de I/O do tipo terminal
- **timers.h** → watchdog timers.
- **stdlib.h** → define tipos, macros e protótipos de funções que provavelmente serão necessárias na compilação de programas C simples.

Arquivos de Cabeçalho Tradicionais (cont.)

- `stdio.h` → raramente usado em arquivos de sistema, mas, assim como o `stdlib.h`, é usada em quase todos os programas de usuário.
- `a.out.h` → define o formato dos arquivos no qual programas executáveis são armazenados no disco.
 - Estrutura `exec` definida aqui e usada pelo PM. Carrega uma nova imagem de programa quando a chamada `exec` é feita.
- `stddef.h` → define macros comumente usadas.

Arquivos de Cabeçalho Tradicionais (cont.)

- `sys/types.h` → define muitos tipos de dados usados pelo MINIX 3.

Type	16-Bit MINIX	32-Bit MINIX
gid_t	8	8
dev_t	16	16
pid_t	16	32
ino_t	16	32

The size, in bits, of some types on 16-bit and 32-bit systems.

Arquivos de Cabeçalho Tradicionais (cont.)

- `sys/sigcontext.h` → define estruturas usadas para preservar e restaurar a operação normal do sistema antes/depois da execução de uma rotina de manipulação de sinal.
 - Usado pelo kernel e PM.
- `sys/stat.h` → define a estrutura retornada pelas chamadas de sistema `stat` e `fstat`, assim com o protótipo destas funções e outras usadas para manipulação de propriedades de arquivos.
 - Usada pelo FS e PM.

Arquivos de Cabeçalho Tradicionais (cont.)

```
struct stat{
    short st_dev;           /* device where i-node belongs */
    unsigned short st_ino;  /* i-node number */
    unsigned short st_mode; /* mode word */
    short st_nlink;         /* number of links */
    short st_uid;           /* user id */
    short st_gid;           /* group id */
    short st_rdev;          /* major/minor device for special files */
    long st_size;           /* file size */
    long st_atime;          /* time of last access */
    long st_mtime;          /* time of last modification */
    long st_ctime;          /* time of last change to i-node */
};
```

Arquivos de Cabeçalho Tradicionais (cont.)

- `sys/dir.h` → define a estrutura de uma entrada de diretório no MINIX.
- `sys/ptrace.h` → define as várias operações possíveis com a chamada de sistema `ptrace`.
- `sys/svrctl.h` → define estruturas de dados e macros usadas pelo `svrctl` (não é uma syscall mas é usado como uma).
 - `svrctl` é usado para coordenar processos de nível de servidor, à medida que o sistema inicia.

Arquivos de Cabeçalho Tradicionais (cont.)

- `sys/select.h` → permite a espera por entrada em múltiplos canais.
 - ex. pseudo terminais esperando por conexões de rede.
- `sys/ioctl.h` → syscall `ioctl` usada para operações de controle de dispositivo.

- Os subdiretórios `include/minix/` e `include/ibm/` contém arquivos de cabeçalho específicos do MINIX 3.
- Arquivos em `include/minix/` necessários para uma implementação do MINIX 3 em qualquer plataforma.

Arquivos de Cabeçalho do MINIX (cont.)

- `config.h` → arquivo para inserção de modificações no MINIX.
 - Incluído nos cabeçalhos mestres para todas as partes do sistema MINIX 3.
 - `minix/sys_config.h` → contém definições de alguns parâmetros. Usado pelo `config.h`.
 - Provavelmente necessário por um programador de sistema. ex. Para desenvolvimento de controladores de dispositivo.
 - `_NR_PROCS` → controla o tamanho da tabela de processo.

Arquivos de Cabeçalho do MINIX (cont.)

- `const.h` → define constantes que não deverão ser mudadas ao compilar um novo kernel e usadas em diversos lugares.
 - `src/kernel/const.h` → definições usadas apenas no kernel.
 - `src/servers/fs/const.h` → definições usadas apenas no sistema de arquivos.
- **OBS.: somente definições que são usadas em mais do que uma parte do MINIX são incluídas em `include/minix/const.h`**

Arquivos de Cabeçalho do MINIX (cont.)

- EXTERN, PRIVATE, PUBLIC e constantes numéricas.
- type.h → contém definições chave e valores numéricos relacionados.
 - Dois diferentes tipos de mapeamento de memória – um para regiões de memória local (espaço de endereçamento de um processo) e áreas de memória remotas (ex. RAM disk)
 - A unidade de memória do MINIX é o click – 1024 bytes.
 - phys_clicks é usado pelo kernel para acessar qualquer elemento de memória em qualquer lugar do sistema.
 - vir_clicks usado por processos além do kernel.

Arquivos de Cabeçalho do MINIX (cont.)

- `vir_clicks` define o tamanho de uma unidade de memória alocada a um processo. Ao usar esta unidade, uma verificação é realizada para assegurar que memória não é acessada fora daquilo que foi atribuído ao processo corrente.
 - Característica maior do modo protegido de processadores Intel, como a família Pentium.
- `sigmsg` → quando um sinal é recebido, o kernel precisa executar um *handler*, ao invés de continuar a execução onde foi interrompido.
- Estrutura `kinfo` → usada para conduzir informação sobre o kernel a outras partes do sistema. O PM usa esta informação quando ajusta parte da tabela de processo.

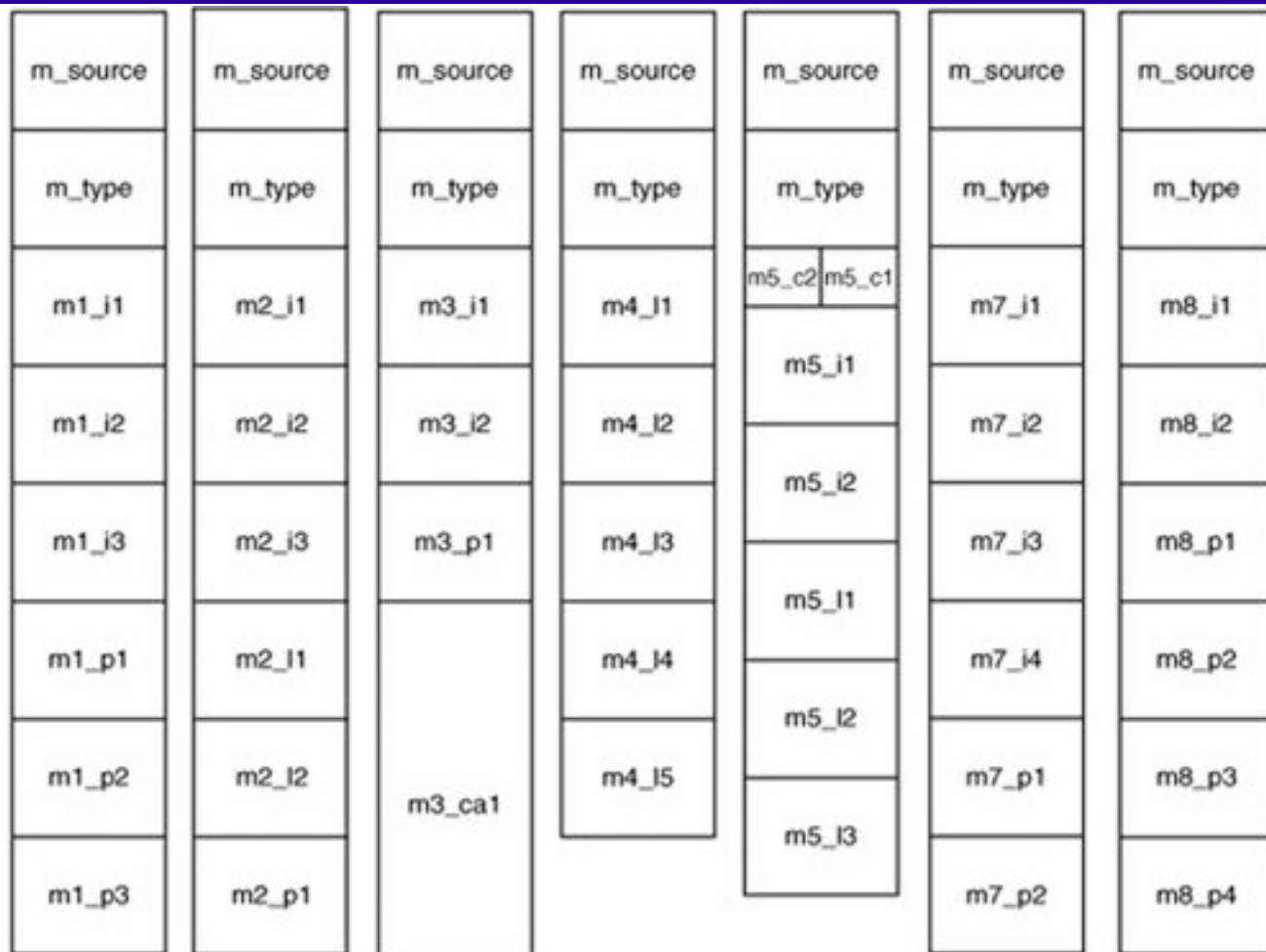
Arquivos de Cabeçalho do MINIX (cont.)

- `ipc.h` → estruturas de dados e protótipos para comunicação interprocessos.
 - Definição de mensagem é crucial.
 - Poderia ser definido como um vetor de qualquer tamanho (em bytes). Boas práticas recomendam a definição como union contendo vários tipos de mensagens possíveis.
 - Sete formatos de mensagem são possíveis (`mess_1` até `mess_8` – `mess_6` está obsoleto).
 - A mensagem é uma estrutura contendo um campo `m_source`, dizendo quem enviou a mensagem, um campo `m_type`, dizendo qual o tipo da mensagem (ex. `SYS_EXEC` para tarefa de sistema) e campos de dados.

Arquivos de Cabeçalho do MINIX (cont.)

- Também define os protótipos para as primitivas de passagem de mensagem. Além das primitivas send, receive, sendrec e notify outras são definidas.

Arquivos de Cabeçalho do MINIX (cont.)



Os sete tipos de mensagem. O tamanho dos elementos da mensagem varia, dependendo do tipo de arquitetura da máquina.

Arquivos de Cabeçalho do MINIX (cont.)

```
O tamanho de um bool é: 1 byte
O tamanho de um char é: 1 byte
O tamanho de um int (modificador short) é: 2 bytes
O tamanho de um int é: 4 bytes
O tamanho de um int (modificador long) é: 8 bytes
O tamanho de um double é: 8 bytes
O tamanho de um double (modificador long) é: 16 bytes
```

sizeof no Mac OS X

```
O tamanho de um bool é: 1 byte
O tamanho de um char é: 1 byte
O tamanho de um int (modificador short) é: 2 bytes
O tamanho de um int é: 4 bytes
O tamanho de um int (modificador long) é: 4 bytes
O tamanho de um double é: 8 bytes
O tamanho de um double (modificador long) é: 12 bytes
```

sizeof no FreeBSD

Arquivos de Cabeçalho do MINIX (cont.)

- `syslib.h` → Contém protótipos para funções de biblioteca C chamadas a partir do SO para acessar outros serviços do SO.
- As funções referenciadas pela `syslib.h` são específicas do MINIX 3 e o port do MINIX 3 para um novo sistema com um compilador diferente, requer o port destas funções de biblioteca.

Arquivos de Cabeçalho do MINIX (cont.)

- `callnr.h` → contém o índice das chamadas de sistema.
- `com.h` → oferece definições comuns usadas para comunicação entre servidores e controladores de dispositivo.
- `devio.h` → define tipos e constantes que suportam o acesso a partir de espaço de usuário a portas de I/O, assim como macros para facilitar o acesso.

Arquivos de Cabeçalho do MINIX (cont.)

- `dmap.h` → define uma estrutura e um array daquela estrutura, chamada `dmap`.
 - Definições das funções de suporte.
 - Números de dispositivo major e minor.
- `keymap.h` → define estruturas usadas para implementar layouts de teclado especializados para caracteres em diferentes linguagens.
- `bitmap.h` → oferece poucas macros para tornar operações como ajuste (*setting*), reajuste (*resetting*) e testes de bit mais simples.

Arquivos de Cabeçalho do MINIX (cont.)

- `partition.h` → informação necessária pelo MINIX 3 para definir uma partição de disco, pelo seu *byte offset* absoluto e tamanho no disco, ou por um cilindro, cabeça, endereço de setor.

Estruturas de Dados de Processo e Arquivos de Cabeçalho

- Código em `src/kernel/`
- Arquivo `kernel.h` → torna possível garantir que todos os arquivos-fonte compartilhem um grande número de definições importantes.
 - 3 macros
 - `_POSIX_SOURCE` → assegura que todos os símbolos requeridos pelo padrão e quaisquer outros que sejam explicitamente permitidos (mas não necessários) estejam visíveis enquanto esconde símbolos adicionais que não sejam oficialmente pertencentes à extensão POSIX.

Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- `_MINIX` → sobrepõe o efeito de `_POSIX_SOURCE` para extensões definidas pelo MINIX 3.
- `_SYSTEM` → define algo de forma diferente enquanto compila o código do sistema.
ex. Mudança de códigos de erro.

Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- Arquivos de cabeçalho master similares são oferecidos em diretórios fonte para outros componentes de sistema. Ex. FS e PM.
- Arquivos de cabeçalho incluídos em kernel.h
 - config.h → análogo ao arquivo de sistema disponível em include/minix/config.h. Deve ser incluído antes de quaisquer outros arquivos de include locais.
 - const.h e type.h em src/kernel/

Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- Os arquivos em `include/minix/` são colocados lá pois são necessários a muitas partes do sistema, incluindo, programas que executam sob o controle do sistema.
- Os arquivos em `src/kernel/` oferecem definições necessárias apenas para a compilação do kernel.
- O FS, PM e outros diretórios de fontes de sistema também contém arquivos `const.h` e `type.h` para definir constantes e tipos necessários somente para aquelas partes do sistema.

Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- Outros arquivos incluídos no cabeçalho master são `proto.h` e `glo.h`, os quais não possuem contra-parte nos principais diretórios de `include`.
 - Possuem contra-parte usada em arquivos de sistema como o FS e o PM.
 - O último arquivo de cabeçalho local incluído em `kernel.h` é outro `ipc.h`
- No início de `kernel/config.h` há uma cláusula `#ifndef ... #define` para evitar problemas se o arquivo for incluído múltiplas vezes.
 - Nota: a macro definida é `CONFIG_H` sem underscore. É diferente da macro `_CONFIG_H` definida em `include/minix/config.h`.

Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- A versão de config.h no kernel reúne em um lugar várias definições que dificilmente precisariam de mudanças.
 - Permite desabilitar chamadas de kernel seletivamente.
 - É possível criar uma versão pequena do MINIX 3 para controlar um instrumento científico ou telefone celular.
 - Define uma macro para converter endereços virtuais relativos a base do espaço de memória kernel para endereços físicos.
 - Mecanismo de segurança → processor status word (PSW) e I/O Protection Level (IOPL) bits definem se é permitido o acesso a interrupções de sistema e I/O.

Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- `type.h` → protótipos e estruturas usadas em qualquer implementação do MINIX 3.
 - ex. `kmessages`, usado para gerar mensagens de diagnóstico do kernel e, `randomness`, usado pelo gerador de números aleatórios.
 - Também contém várias definições de tipo dependentes de máquina.
- `proto.h` → oferece protótipos de todas as funções que devem ser conhecidas fora do arquivo no qual são definidas.

Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- Alguns destes protótipos são dependentes de sistema, incluindo manipuladores de interrupção e exceção e funções que são escritas em linguagem assembly.
- `glo.h` → variáveis globais do kernel.
 - Algumas das estruturas de informação aqui são usadas no *startup*.
 - Variáveis relacionadas ao controle de processo e execução do kernel.
 - `Prev_ptr`, `proc_ptr`, and `next_ptr` apontam para as entradas da tabela de processo do processo anterior, corrente e próximo a ser executado

Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- Bill_ptr também aponta para uma entrada na tabela de processo; mostra qual processo está sendo temporizado quanto a *clock ticks* usados.

Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- `proc.h` (não incluído em `kernel.h`) → define a tabela de processo do kernel.
 - O estado completo de um processo é definido pelos dados do processo em memória, juntamente com a informação em seu slot na tabela de processo.
 - Os conteúdos dos registradores da CPU são armazenados aqui quando um processo não está executando, e restaurados quando a execução continua.
 - É o que torna possível a ilusão de que múltiplos processos estão executando simultaneamente e interagindo.
 - O tempo gasto pelo kernel salvando e restaurando o estado do processo durante cada chaveamento de contexto é necessário.

Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- Cada *slot* na tabela de processo é definido como uma struct proc.
 - Cada entrada contém espaço para os registradores do processo, ponteiro de pilha, estado, mapa de memória limite de pilha, id do processo, *accounting*, *alarm time* e *message info*.
 - A primeira parte de cada tabela de processo é uma estrutura `stackframe_s`. Um processo que já está na memória é colocado para executar pela carga do ponteiro de pilha com o endereço de sua tabela de processo e carga de todos os registradores da CPU (*pop*) desta estrutura.

Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- Cada processo possui um ponteiro para uma estrutura priv em seu slot na tabela de processo.
 - Esta estrutura define códigos fonte e destinos de mensagem para o processo e muitos outros privilégios.
- Cada *slot* na tabela de processo oferece espaço para informação que pode ser necessária ao kernel.
 - O campo `p_max_priority`, informa em qual fila de escalonamento o processo deve ser enfileirado quando está pronto para executar pela primeira vez. Lembrete: a prioridade de um processo pode ser reduzida caso ele previna que outros processos sejam executados.

Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- O campo `p_priority` (inicialmente igual a `p_max_priority`) determina a fila usada a cada vez quando o processo está pronto.
- O tempo usado por cada processo é guardado em duas variáveis `clock_t`.
- `p_nextready`, é usado para ligar processos juntos nas filas de escalonamento.
- Demais campos mantém informação relacionada a mensagens entre processos.
 - O método de rendezvous para passagem de mensagem é possível graças a espaço de armazenamento reservado. `p_getfrom`, `p_sendto`, `p_messbuf`.

Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- O penúltimo campo em cada slot de tabela de processo é `p_pending`, um bitmap usado para registrar os sinais que ainda não foram passados ao PM (ele não está esperando por uma mensagem).
- O último campo em uma tabela de processo é um vetor de caracteres `p_name` usado para guardar o nome do processo. Não é um campo necessário para gerenciamento de processo no kernel. Útil para geração de informação de debugging.
- Definição do número de filas de escalonamento e valores permitidos para o campo `p_priority`. Na versão atual, processos de usuário podem ter acesso a fila de maior prioridade. O valor de `MAX_USER_Q` deve ser ajustado para uma menor prioridade.

Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- Macros que permitem que endereços de partes importantes da tabela de processo sejam definidas como constantes em tempo de compilação, para prover acesso rápido em tempo de execução.
- Mais macros para cálculos e testes em tempo de execução.
- Várias macros são definidas para acelerar o acesso (vetor de ponteiros para os elementos da tabela de processo)

Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- `priv.h` → estrutura de privilégios de sistema, `priv`.
 - Similar a estrutura do `proc.h`
 - A estrutura `priv` não é parte da tabela de processo. Cada slot na tabela de processo possui um ponteiro para uma instância dessa estrutura.
 - Somente processos de sistema possuem cópias privadas; processos de usuário apontam para a mesma cópia.

Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- Processo para assegurar que NR_SYS_PROCS foi definido como maior do que o número de processos na imagem de boot.
- A tecla F4 dispara um *debug dump* que mostra algumas das informações na tabela de privilégios.

```
--nr-   -id-   -name-   -flags-   -traps-   -ipc_to mask -----
(-4)   (01)   IDLE   P-BS-   -----   00000000 00001111
[-3]   (02)   CLOCK  ---S-   --R--   00000000 00001111
[-2]   (03)   SYSTEM  ---S-   --R--   00000000 00001111
[-1]   (04)   KERNEL  ---S-   -----   00000000 00001111
  0    (05)   pm      P--S-   ESRBN   11111111 11111111
  1    (06)   fs      P--S-   ESRBN   11111111 11111111
  2    (07)   rs      P--S-   ESRBN   11111111 11111111
  3    (09)   memory  P--S-   ESRBN   00110111 01101111
  4    (10)   log     P--S-   ESRBN   11111111 11111111
  5    (08)   tty     P--S-   ESRBN   11111111 11111111
  6    (11)   driver  P--S-   ESRBN   11111111 11111111
  7    (00)   init    P-B--   E--B-   00000111 00000000
```

Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- Os flags significam: P: preemptable, B: billable, S: system.
- As traps significam E: echo, S: send, R: receive, B: both, N: notification.
- O bitmap possui um bit para cada um dos (32) processos de sistema permitidos (NR_SYS_PROCS). **A ordem corresponde ao campo id.** Todos os processos de usuário compartilham o id 0, o qual é a posição binária mais a esquerda.
- **O bitmap mostra que processos de usuário como o init podem enviar mensagens somente ao PM, FS e RS e devem usar sendrec. Os servidores e drivers mostrados na figura podem usar quaisquer das primitivas de ipc e todos menos a memória podem enviar mensagens a quaisquer outros processos**

Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- `protect.h` → detalhes arquiteturais de processadores intel que suportam modo protegido.
- As partes centrais do kernel, envolvidas em interrupções e chaveamento de contexto executam com `INTR_PRIVILEGE`. Todo endereço na memória e todos os registradores na CPU podem ser acessados por um processo com este nível de privilégio.
- As tarefas executam em nível `TASK_PRIVILEGE`, os quais permitem acesso de I/O, mas não para usar instruções que modifiquem registradores especiais, como aqueles que apontam para tabelas de descritor.

Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- Servidores e processos de usuário executam em nível USER_PRIVILEGE. Processos executando neste nível não estão aptos a executar certas instruções, ex, aquelas que acessam portas de I/O, mudança de atribuições de memória, ou mudança de níveis de privilégio.

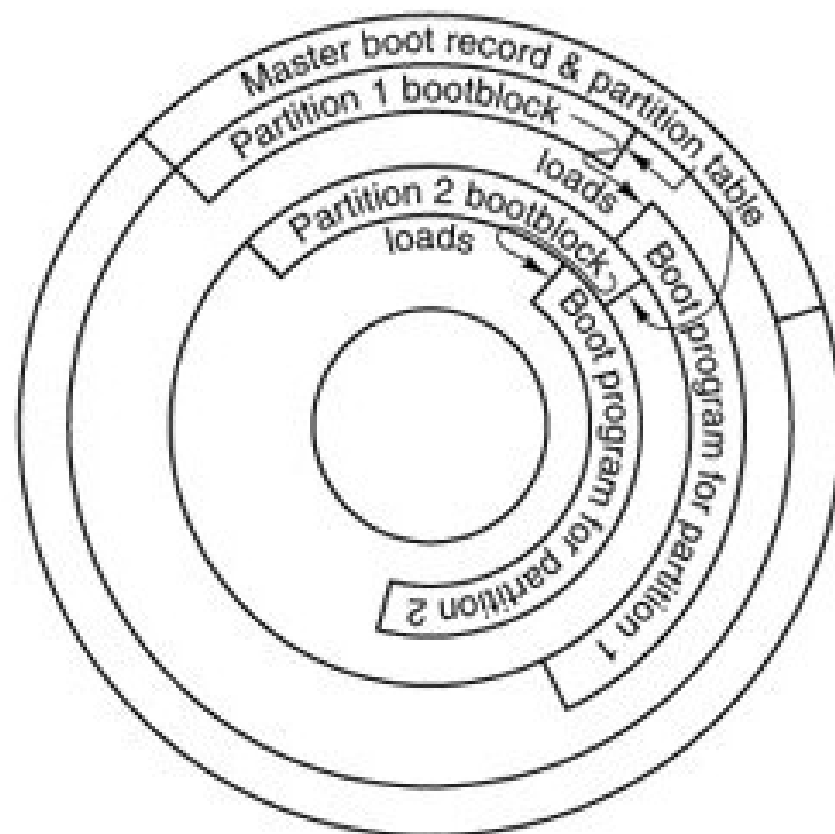
Estruturas de Dados de Processo e Arquivos de Cabeçalho (cont.)

- `ipc.h` → define várias constantes usadas para comunicação interprocessos.
- `table.c` → processos de usuário, como filhos do `init`, executam por 8 ticks de clock. Tarefas de `CLOCK` e `SYSTEM` podem executar por 64 ticks de clock se for necessário.

Bootstrapping do MINIX 3



(a)



(b)

Estruturas de disco usadas para bootstrapping. (a) Disco não particionado. Primeiro setor é o bootblock (b) Disco particionado. Primeiro setor é o master boot record.

- Reserva-se o primeiro bloco de 1K de cada dispositivo de disco como um *bootblock* (similar ao UNIX), mas somente um setor de 512 bytes é carregado pelo *boot loader* da ROM ou master boot sector.
 - 512 bytes disponíveis para salvar *settings*.

- Quando o sistema é iniciado, o hardware (um programa na ROM) lê o primeiro setor do disco, copia o conteúdo para uma localização fixa na memória, e executa o código.
 - Em disco não particionado, o primeiro setor é um *bootblock* o qual carrega o programa de boot.
 - Discos particionados, e o programa no primeiro setor se realocam a uma região de memória diferente, então lêem a tabela de partição, carregada a partir do primeiro setor. Então, carrega e executa o primeiro setor da partição ativa.

Bootstrapping do MINIX 3 (cont.)

- Geralmente apenas uma partição é marcada como ativa.
- Uma partição MINIX 3 possui a mesma estrutura de um disco não particionado.
 - O código do *bootblock* é o mesmo.
 - Como o programa MBR se realoca, o código do *bootblock* pode ser escrito para executar no mesmo endereço de memória em que o MBR foi carregado originalmente.
- Variações podem existir. Uma partição pode conter subpartições. Neste caso, o primeiro setor da partição será outro MBR contendo a tabela de partição para as subpartições.
 - Eventualmente o controle será passado a um setor de boot, o primeiro setor em um dispositivo que não seja subdividido.

Bootstrapping do MINIX 3 (cont.)

- O setor de boot do MINIX 3 é modificado no momento que é escrito no disco por um programa especial chamado installboot.
 - Escreve o setor de boot e passa o endereço de um arquivo chamado boot na partição (ou subpartição).
 - O caminho padrão para o programa de boot é em /boot/boot.
 - Poderia estar em qualquer lugar. Necessário saber exatamente os setores de disco.
 - Boot é o carregador secundário para o MINIX 3.
 - É um programa monitor que permite ao usuário mudar, ajustar e salvar vários parâmetros.

Bootstrapping do MINIX 3 (cont.)

- Não é parte do sistema operacional.
Mesmo assim, pode usar estrutura de sistema de arquivos para encontrar a imagem do SO. Procura pelo arquivo com o nome especificado em image = boot, o qual geralmente é /boot/image
- As regiões de memória disponíveis para carga do monitor de boot e programas componentes do MINIX 3 dependerão do hardware.
 - Algumas arquiteturas podem requerer ajuste de endereçamento interno dentro do código executável para corrigir o endereço de carga do programa.
Desnecessário em plataforma Intel.

Bootstrapping do MINIX 3 (cont.)

- Os parâmetros de boot, devem ser comunicados a partir do boot monitor para o SO.

```
rootdev=904
ramimagedev=904
ramsize=0
processor=686
bus=at
video=vga
chrome=color
memory=800:92540,100000:3DF0000
label=AT
controller=c0
image=boot/image
```

- No exemplo, perceba o parâmetro de memória.
 - Neste caso, indica que o monitor de boot determinou que existem dois segmentos de memória disponíveis para uso pelo MINIX 3.
 - Um começa no endereço hexadecimal 800 (decimal 2048) e possui um tamanho de 0x92540 (decimal 599,360) bytes; o outro começa em 100000 (1,048,576) e contém 0x3df00000 (64,946,176) bytes. Típico de todos os barramentos, menos dos Pcs mais antigos.

Inicialização do Sistema

- A primeira parte do MINIX 3 a executar é escrita em linguagem assembly
 - Diferentes arquivos-fonte devem ser usados para o compilador de 16 bits e 32 bits.
 - A versão de 32 bits do código de inicialização está em `mpx386.s`. A alternativa para sistemas de 16 bits é o `mpx88.s`.
 - Ambos incluem suporte a linguagem assembly para outras operações de kernel de baixo nível.
 - A seleção é feita automaticamente em `mpx.s`.

Inicialização do Sistema (cont.)

```
#include <minix/config.h>
#if_WORD_SIZE == 2
#include "mpx88.s"
#else
#include "mpx386.s"
#endif
```

Como arquivos fonte alternativos em linguagem assembly são selecionados.

- O *startup* do MINIX 3 envolve várias transferências de controle entre as rotinas de linguagem assembly em `mpx386.s` e rotinas em linguagem C nos arquivos `start.c` e `main.c`.

- Uma vez que o processo de bootstrap carregou o SO na memória, o controle é transferido para o label MINIX (em mpx386.s).
 - A primeira instrução é um jump sobre alguns bytes de dados.
 - flags do boot monitor (usadas quando o boot monitor carregou o kernel na memória).
 - Localizadas aqui pois é um endereço conhecido.
 - Usadas pelo *boot monitor* para identificar várias características do kernel. ex. Sistema de 32 ou 16 bits.

Inicialização do Sistema (cont.)

- O monitor passa vários parâmetros ao MINIX 3 (coloca-os na pilha).
 - O monitor insere (push) o endereço da variável aout (mantém o endereço de um array de informação do cabeçalho dos programas componentes da imagem de boot).
 - Ele insere o tamanho e o endereço dos parâmetros de boot. Quantidades de 32 bits.
 - endereço do código do segmento do monitor e o ponto de retorno dentro do monitor quando o MINIX 3 termina.
 - Quantidades de 16 bits, considerando que o monitor opera em modo protegido de 16 bits.

Inicialização do Sistema (cont.)

- As primeiras instruções em mpx386.s convertem o ponteiro de pilha de 16 bits usado pelo monitor em um valor de 32 bits para uso em modo protegido.
- A instrução `mov ebp, esp` copia o valor do ponteiro de pilha para o registrador `ebp` register, de modo que possa ser usado com *offsets* para recuperar da pilha os valores colocados pelo monitor.
 - Nota: como a pilha cresce para baixo em processadores Intel, 8 (`ebp`) refere-se a um valor inserido (push) posteriormente a um valor inserido localizado em 12 (`ebp`).
- O código assembly deve ajustar uma estrutura de pilha (*stack frame*) para oferecer o ambiente adequado para que o código compilado C, possa copiar tabelas usadas pelo processador para definir segmentos de memória e ajustar vários registradores.

- O processo de inicialização continua pela chamada da função `cstart` (em `start.c`).
- Os primeiros passos são:
 - Ajustar os mecanismos de proteção da CPU e tabelas de interrupção.
 - Copiar os parâmetros de boot para a memória do kernel e verificá-los (`scan`), usando a função `get_value` para buscar nomes de parâmetros e strings de valor correspondente.
 - Este processo determina o tipo de display de vídeo, tipo de processador, tipo de barramento, se está e modo de 16 bits e o modo de operação do processador (real ou protegido).
 - Essa informação é armazenada em variáveis globais, para acesso quando necessário por qualquer parte do código do kernel.

Inicialização do Sistema (cont.)

- Main (em main.c), completa a inicialização e então inicia a execução normal do sistema.
 - A maior parte do código main é dedicada ao ajuste da tabela de processo e tabela de privilégios.
 - Quando as primeiras tarefas e processos são escalonadas, seus mapas de memória, registradores e informação de privilégio serão ajustadas corretamente.
 - Configura o hardware de controle de interrupção pela chamada a `intr_init`.
 - Feito aqui pois é necessário conhecer o tipo de máquina.
 - `intr_init` assegura que quaisquer interrupções que ocorrem antes da inicialização esteja completa, não possuam efeito.

Inicialização do Sistema (cont.)

- Uma vez que a tabela de processo tenha sido inicializada para todas as tarefas, os servidores e init, o sistema está quase pronto.
 - A variável `bill_ptr` informa qual processo é taxado pelo tempo de processamento; precisa ter um valor inicial e IDLE é uma escolha apropriada.
 - Agora o kernel está pronto para começar a funcionar normalmente a desempenhar suas funções de controle e escalonamento da execução de processos (ciclo de vida padrão).
- O procedimento de inicialização foi rudimentarmente discutido. Para maiores informações veja a seção 2.6 do livre “Projeto e Implementação de Sistemas Operacionais de A.S. Tanenbaum.

Manipulação de Interrupção no MINIX

- Hardware de interrupção é dependente de sistema.
 - Qualquer sistema deve ter funcionalidades equivalentes àquelas descritas para CPUs de 32 bits Intel.
- Interrupções geradas por dispositivos de hardware são sinais elétricos e são manipuladas por um controlador de interrupção.
- Um circuito integrado pode ler estes sinais e para cada um gerar um padrão de dados único no barramento de dados do processador.

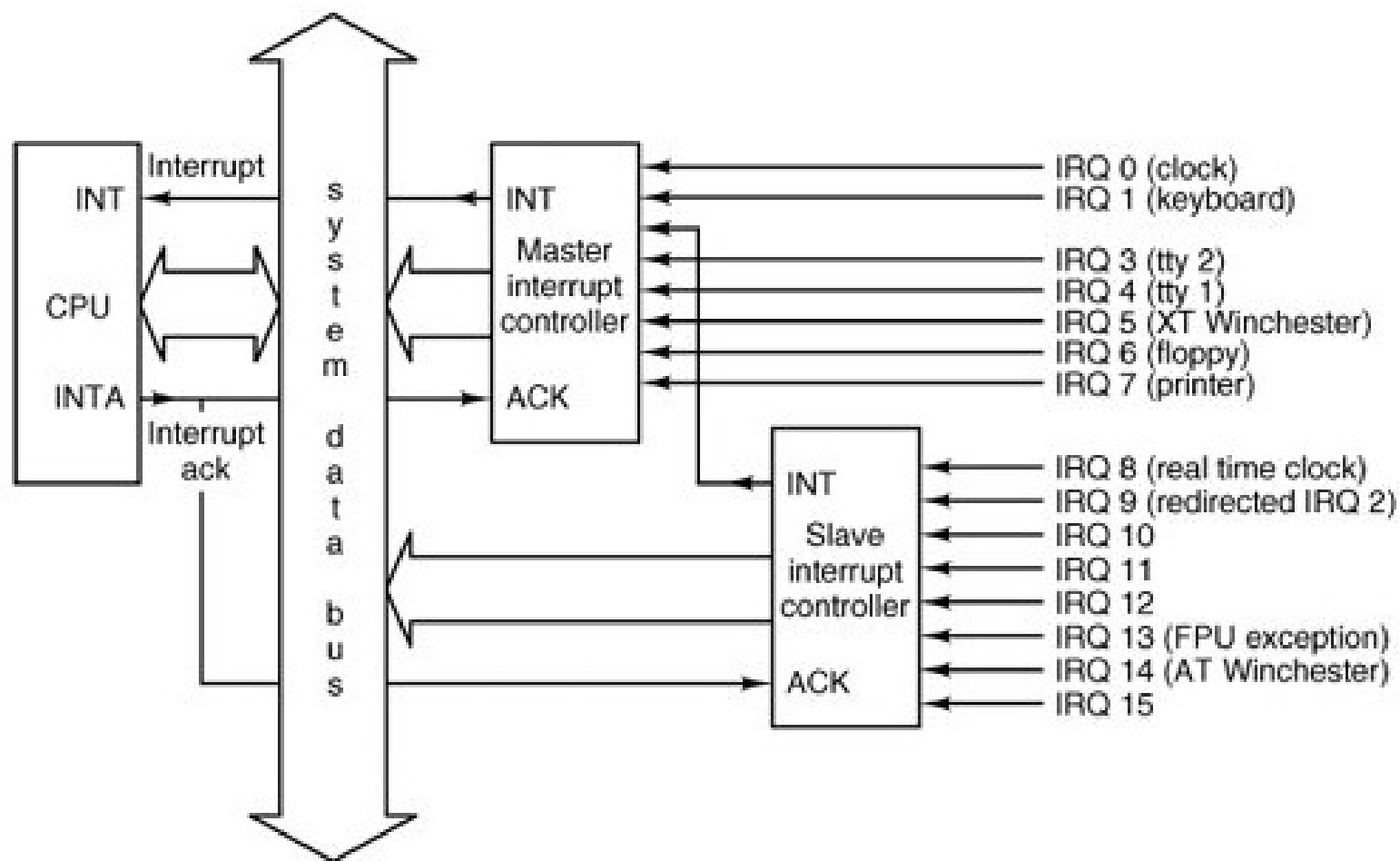
Manipulação de Interrupção no MINIX (cont.)

- **Necessário:** o processador possui apenas uma entrada para ler todos estes dispositivos. Não consegue diferenciar qual dispositivo precisa de serviço.
- Pcs usando processadores de 32 bits possuem normalmente dois chips controladores.
 - Cada um pode manipular 8 entradas, porém um deles é escravo e alimenta sua saída a uma das entradas do mestre. Logo, consegue identificar 15 dispositivos.

Manipulação de Interrupção no MINIX (cont.)

- Algumas das 15 entradas são dedicadas.
 - ex. a entrada de clock, IRQ 0. não possui sockets para ligar quaisquer outros dispositivos. Outras possuem sockets e podem ser usadas para quaisquer dispositivos.

Manipulação de Interrupção no MINIX (cont.)



Manipulação de Interrupção no MINIX (cont.)

- No MINIX 3, os campos de maior atenção em cada um dos *interrupt gate descriptors* apontam para o segmento de código executável e o endereço de início.
 - A CPU executa o código apontado pelo descritor selecionado.

Manipulação de Interrupção no MINIX (cont.)

- Quando a CPU receber uma interrupção enquanto estiver executando um processo, ela ajusta uma nova pilha para usar durante o serviço de interrupção.
 - A localização desta pilha é determinada por uma entrada no *Task State Segment* (TSS).
 - Tal estrutura existe para o sistema como um todo, inicializada pela chamada `prot_init` de `cstart` e modificada enquanto cada processo é iniciado.

Manipulação de Interrupção no MINIX (cont.)

- O efeito é que a nova pilha criada pela interrupção sempre começa no final da estrutura `stackframe_s` dentro da entrada na tabela de processo do processo interrompido.
- A CPU automaticamente insere (push) vários registradores chave na nova pilha, incluindo aqueles necessários para restaurar a pilha do processo interrompido e seu contador de programa.
- Quando o código do manipulador de interrupção começa a executar, ele usa esta área na tabela como sua pilha, e muito da informação necessária para retornar para o processo interrompido já estarão armazenadas.

Manipulação de Interrupção no MINIX (cont.)

- O manipulador de interrupção insere os conteúdos de registradores adicionais, preenchendo a estrutura da pilha, e então chaveia para uma pilha oferecida pelo kernel enquanto realiza o que for necessário para servir a interrupção.

Manipulação de Interrupção no MINIX (cont.)

- A terminação de um serviço de interrupção é feita pelo chaveamento da pilha do kernel de volta a estrutura do frame na tabela de processo, explicitamente removendo os registradores adicionais e executando a instrução `iretd` (return from interrupt).
 - `iretd` restaura o estado que existiu antes de uma interrupção, restaurando os registradores que foram inseridos pelo hardware e chaveando de volta a pilha que estava em uso antes da interrupção.
- Assim, uma interrupção pára um processo e, ao completar, o serviço de interrupção reinicia um processo, possivelmente diferente do que foi mais recentemente parado.

Manipulação de Interrupção no MINIX (cont.)

- A CPU desabilita todas as interrupções ao receber uma interrupção. Isto garante que nada pode ocorrer para causar um overflow da estrutura da pilha dentro de um processo.
- Um mecanismo existe para permitir que um manipulador de exceção execute quando a pilha do kernel está em uso.
 - Exceção: uma resposta a um erro detectado pela CPU.

Manipulação de Interrupção no MINIX (cont.)

- Uma exceção é similar a uma interrupção e exceções não podem ser desabilitadas.
- A CPU insere os registradores essenciais necessários para inserir o código interrompido na pilha existente.
- Uma exceção não deve ocorrer enquanto o kernel está executando, podendo resultar em *panic*.

Manipulação de Interrupção no MINIX (cont.)

- No caso usual, o código interrompido é menos privilegiado do que o código do serviço de interrupção.
 - Neste caso o mecanismo mais elaborado, usando o TSS e uma nova pilha é empregado.
- Duas tabelas são declaradas em `glo.h` e usadas aqui.
 - `_Irq_handlers` contém a informação de *hook*, incluindo endereços de rotinas de *handler*.

Manipulação de Interrupção no MINIX (cont.)

- Um valor diferente de zero em `_irq_actids` mostra que o serviço de interrupção para esta IRQ não está completo. Assim, o controlador de interrupção é manipulado para prevenir que respondesse a outra interrupção da mesma linha de IRQ.
- `_Intr_handle` rastreia (scan) uma lista ligada de estruturas que mantém, entre outras coisas, o endereço de funções a serem chamadas para lidar com uma interrupção para um dispositivo e os números de processo dos drivers de dispositivo.
 - É uma lista ligada pois uma única linha de IRQ pode ser compartilhada com vários dispositivos.

Manipulação de Interrupção no MINIX (cont.)

- O *handler* para cada dispositivo é supostamente para testar se o dispositivo precisa de serviço.
- O código do *handler* é pretendido para executar rapidamente. Se não há trabalho a fazer ou o serviço de interrupção é completado imediatamente, o *handler* retorna TRUE.
 - Este mecanismo assegura que nenhum dos *handlers* na cadeia pertencente a um IRQ serão ativados, até que todos os *device drivers* para os quais eles respondem tenham completado seu trabalho.
 - É necessário algum mecanismo para reabilitar um IRQ.
 - Função *enable_irq*

Manipulação de Interrupção no MINIX (cont.)

- Este procedimento se aplica, em sua forma mais simples, somente à tarefa de clock.
Este é o único dispositivo orientado a interrupção que é compilado no binário do kernel.

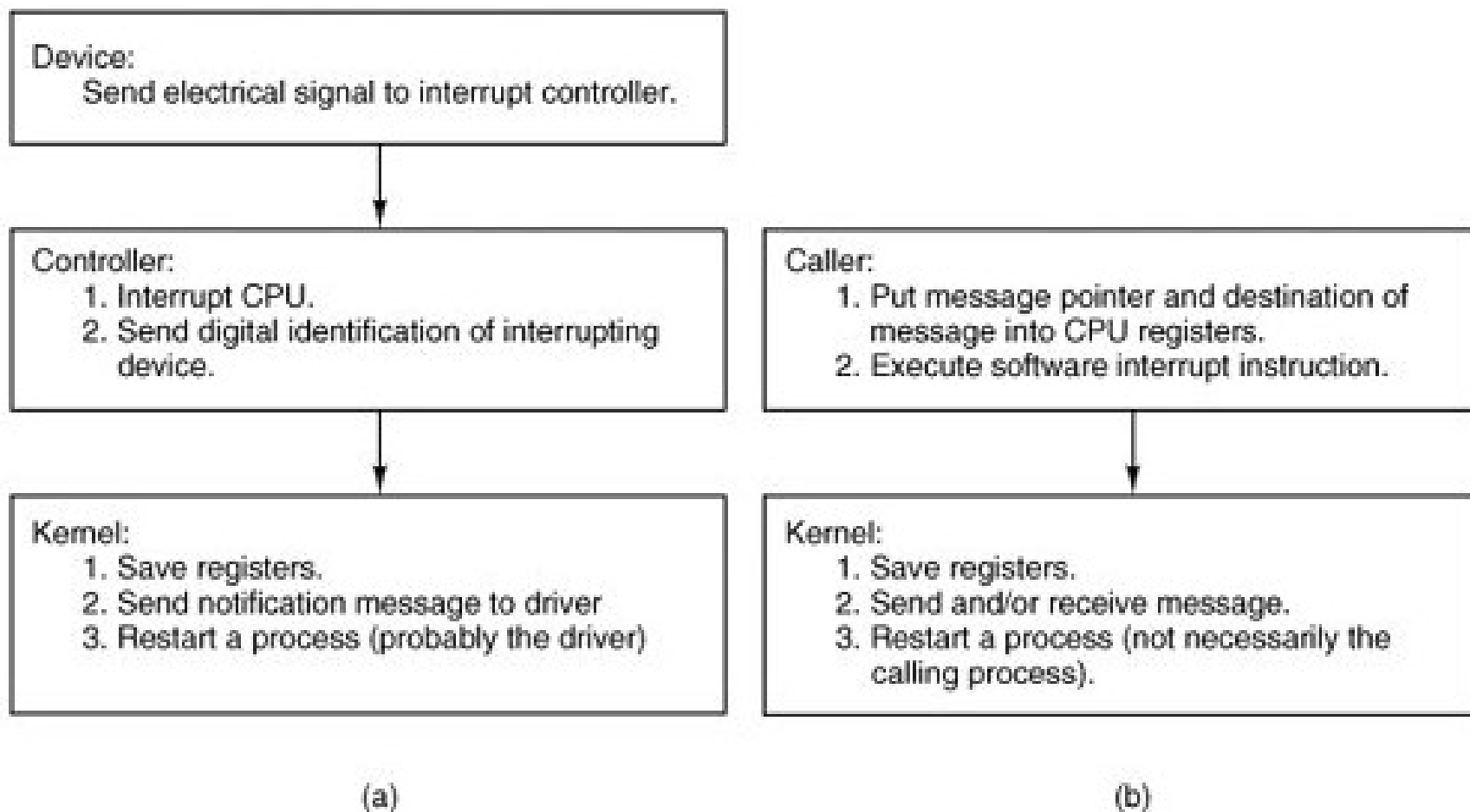
Manipulação de Interrupção no MINIX (cont.)

- Para *device drivers* de espaço de usuário, o endereço de um *handler*, no caso, *generic_handler*, é armazenado em uma lista ligada.
 - O código fonte para esta função está nos arquivos de tarefa de sistema.
 - Compilada juntamente com o kernel e como o código é executado em resposta a uma interrupção, este não pode ser considerado parte da tarefa de sistema.
 - Caso uma interrupção pudesse ocorrer enquanto o código do kernel estiver executando, a pilha do kernel estaria em uso e uma operação *save* deixaria o endereço de retorno na pilha do kernel.
 - O que o kernel estava fazendo anteriormente continuaria após o retorno ao final de *hwint_master*.

Manipulação de Interrupção no MINIX (cont.)

- Mecanismo necessário para a manipulação de exceções.
- Quando todas as rotinas do kernel envolvidas na resposta a exceção são completadas, `_restart` será finalmente executada.
- Em resposta a uma exceção, enquanto executar código do kernel, é quase certo que um processo diferente do que foi interrompido por último será colocado para executar.
- A resposta a uma exceção no kernel é um “panic” seguida de uma tentativa para shut down do sistema com o mínimo de dano possível.
- `hwint_slave` é similar a `hwint_master`, exceto que este deve reabilitar ambos, controlador mestre e escravo, considerando que ambos são desabilitados pela recepção de uma interrupção pelo escravo.

Manipulação de Interrupção no MINIX (cont.)



(a) Interrupção de hardware. (b) Chamada de sistema.



O *restart* é o ponto comum alcançado após o *startup* do sistema, interrupções ou chamadas de sistema. Não mostrado neste diagrama estão interrupções que ocorrem enquanto o kernel está executando (exceções).

Comunicação Interprocessos no MINIX 3

- Processos se comunicam por meio de mensagens usando o princípio de rendezvous.
- Quando um processo realiza um send, a camada mais baixa do kernel verifica se o destino está esperando por uma mensagem do emissor (ou ANY).
 - Nesse caso, a mensagem é copiada do buffer do emissor e ambos os processos são marcados como executáveis.
 - Se o destino não está esperando por uma mensagem, marca-se o emissor como bloqueado e este é colocado em uma fila de processos esperando para enviar ao receptor.

Comunicação Interprocessos no MINIX 3 (cont.)

- Quando um processo realiza um *receive*, o kernel verifica se algum processo está em fila aguardando para enviar a ele.
 - Se sim, a mensagem é copiada do emissor bloqueado para o receptor, e ambos são marcados como executáveis.
 - Se nenhum processo é enfileirado tentando enviar a ele, o receptor bloqueia até que uma mensagem chegue.

Comunicação Interprocessos no MINIX 3 (cont.)

- Algumas vezes, o método de rendezvous não é bom o suficiente.
 - Componentes de sistema executando como processos totalmente separados.
- A primitiva *notify* é oferecida para estas ocasiões.
 - Envia uma mensagem simples.
 - Notifica o receptor de sua origem. Informações adicionais podem ser adquiridas posteriormente. O destino pode enviar uma mensagem a origem da notificação e solicitar mais informações.

Comunicação Interprocessos no MINIX 3 (cont.)

- O emissor não é bloqueado se o destino não está esperando por uma mensagem.
- A próxima vez que um destinatário executar um *receive*, notificações pendentes são entregues antes de mensagens normais.
- Notificações podem ser usadas em situações usando mensagens ordinárias que poderiam causar *deadlocks*.
 - ex. $A \rightarrow B$ e $B \rightarrow A$ (ambos não executariam *receive*).

Comunicação Interprocessos no MINIX 3 (cont.)

- O código de alto nível para comunicação interprocessos é encontrado em `proc.c`
- O trabalho do kernel é traduzir uma interrupção de hardware ou uma interrupção de software em uma mensagem.
- Duas macros úteis são definidas.
 - `BuildMess` → usada somente para construção de mensagens usadas pelo `notify`.
 - `CopyMess` → interface amigável ao programador para a rotina em assembly `cp_mess` em `klib386.s`.

Comunicação Interprocessos no MINIX 3 (cont.)

- A primeira função em `proc.c` é `sys_call`.
 - Converte uma interrupção de software (instrução `SYS386_VECTOR`, pela qual uma `syscall` é iniciada) em uma mensagem.
 - A chamada pode requerer envio ou recebimento ou ambos para uma dada mensagem.
 - As funções `mini_send`, `mini_rec`, e `mini_notify` são o coração do mecanismo de passagem de mensagem normal do MINIX 3.
 - `mini_send` → possui 3 parâmetros – o chamador, o processo destino e um ponteiro para o buffer no qual a mensagem está.
 - Verifica se o chamador e destino não estão tentando executar o *send* entre si.

Comunicação Interprocessos no MINIX 3 (cont.)

- `mini_rec` → chamado por `sys_call` quando seu argumento é `RECEIVE` ou `BOTH`.
 - Notificações possuem uma prioridade mais alta do que mensagens ordinárias.
 - Uma notificação nunca será a resposta correta a um *send*.
 - verifica se existem notificações pendentes somente se a flag `SENDREC_BUSY` não está ajustada.
 - Se uma notificação for encontrada ela é marcada como não pendente e entregue.
 - Usa `BuildMess` e `CopyMess` definidas em `proc.c`.

Comunicação Interprocessos no MINIX 3 (cont.)

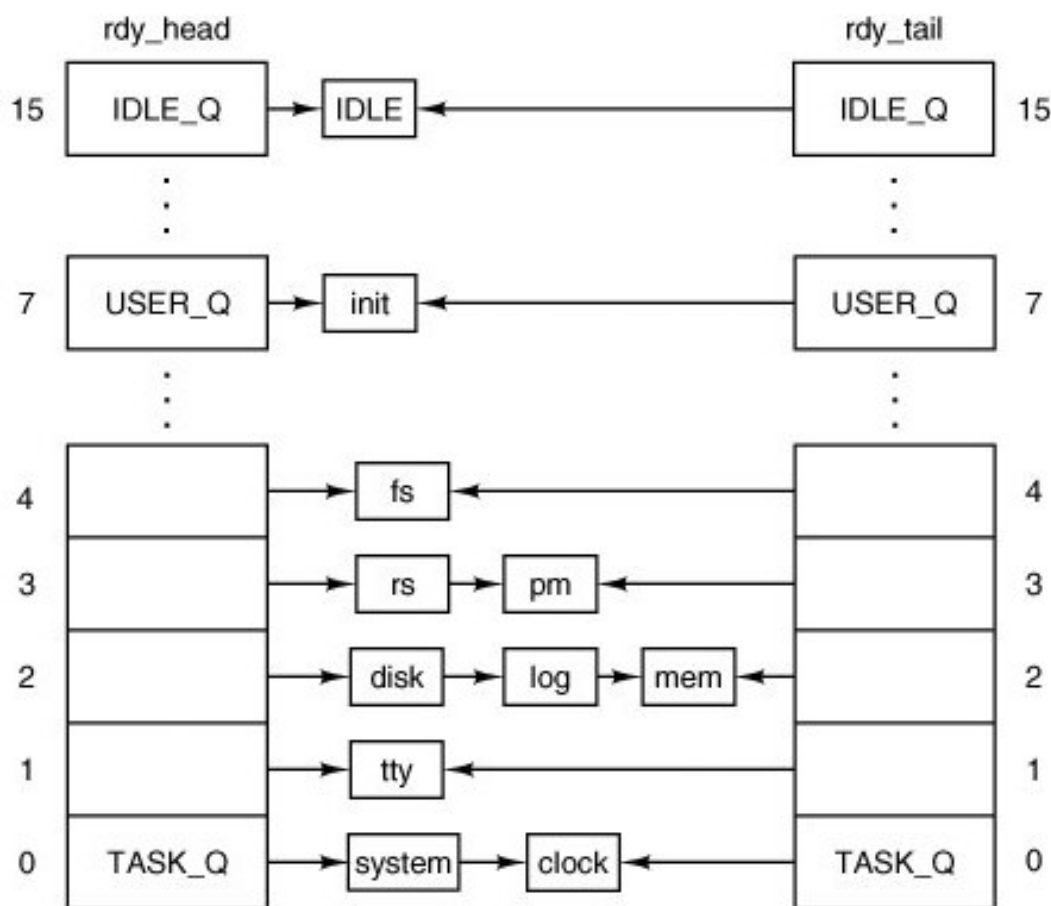
- `mini_notify` → Usado para efetuar uma notificação.
 - Similar a `mini_send`.
 - Se o destinatário de uma mensagem é bloqueado e espera pelo *receive*, a notificação é gerada por `BuildMess` e entregue.
 - Se o receptor não está esperando um bit é ajustado no mapa `s_notify_pending`, o qual indica se uma notificação está pendente e identifica o emissor.
 - O emissor continua posteriormente. Se nova notificação de um mesmo destinatário é necessária antes de uma notificação pendente o bitmap é sobrescrito.
 - Este projeto elimina a necessidade por gerenciamento de buffer enquanto oferece passagem de mensagem assíncrona.

Comunicação Interprocessos no MINIX 3 (cont.)

- O `mini_notify` pode ser interrompido devido a uma interrupção. Pode ser que as tarefas de clock ou sistema precisem desta função. Assim, a notificação pode ser enviada quando as interrupções não estiverem desabilitadas.

- Algoritmo de escalonamento multinível.
- Processos possuem prioridades iniciais que são relacionadas a estrutura multinível.
- `rdy_head` possui uma entrada para cada fila, com a entrada apontando para o processo na cabeça da fila.
- `rdy_tail` é um vetor cujas entradas apontam para o último processo em cada fila.

Escalonamento no MINIX 3 (cont.)



O escalonador mantém 16 filas, uma por nível de prioridade.

Escalonamento no MINIX 3 (cont.)

- Escalonamento é round robin em cada fila.
- Se um processo usa seu quantum ele é movido para o final da fila e recebe um novo quantum.
- Se um processo é bloqueado ele é acordado e colocado na cabeça da fila. Se possui alguma parte do quantum sobrando ao bloquear.
- A existência de `rdy_tail` torna simples a adição de um processo no fim da fila.
- Somente processos executáveis são colocados na fila.

Escalonamento no MINIX 3 (cont.)

- O processo IDLE sempre está pronto, e está na fila de mais baixa prioridade.
- Se todas as filas de prioridade mais alta estão vazias IDLE é executado.

- Em uma chamada de sistema fork, como avisar o kernel que o processo foi criado?
 - PM → kernel?
- Tarefa da system task.
 - Apesar de ser compilada no kernel, em realidade é um processo e é escalonado como tal.
- Assim...
 - System call é usado para se referir a chamadas para todos os serviços fornecidos pelo kernel.

Tarefa de Sistema no MINIX 3 (cont.)

- UNIX → padrão POSIX descreve as chamadas de sistema disponíveis a processos → acesso a funções de kernel.
- MINIX → componentes do sistema operacional são executados em espaço de usuário – possuem privilégios especiais (PM, FS, RS, etc).
 - System task recebe requisições para serviços do kernel → kernel calls.
 - Kernel calls não podem ser feitas por processos de usuário.

Tarefa de Sistema no MINIX 3 (cont.)

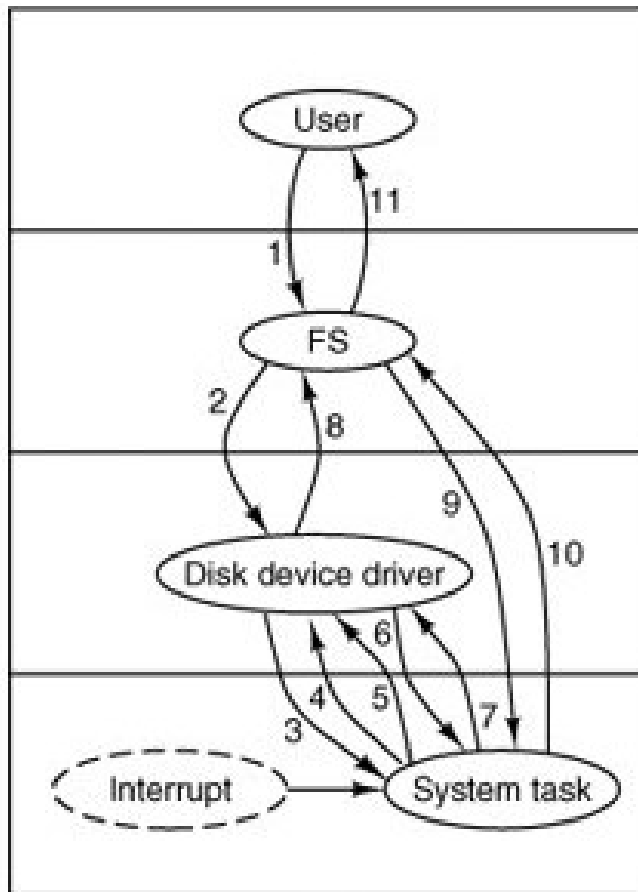
- Processo de usuário (system call) → servidor (kernel call) → tarefa de sistema
 - ex. fork() → PM (sys_fork) → tarefa de sistema
 - Primitivas de mensagens também poderiam ser colocadas como uma terceira categoria de chamadas (send, receive, notify)

Tarefa de Sistema no MINIX 3 (cont.)

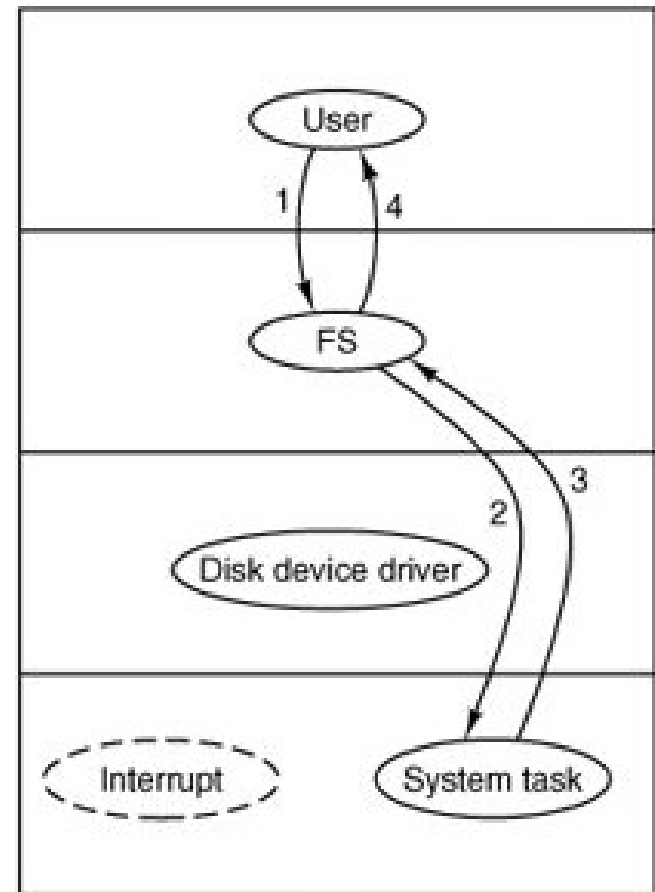
Message type	From	Meaning
sys_fork	PM	A process has forked
sys_exec	PM	Set stack pointer after EXEC call
sys_exit	PM	A process has exited
sys_nice	PM	Set scheduling priority
sys_privctl	RS	Set or change privileges
sys_trace	PM	Carry out an operation of the PTRACE call
sys_kill	PM, FS, TTY	Send signal to a process after KILL call
sys_getksig	PM	PM is checking for pending signals
sys_endksig	PM	PM has finished processing signal



Tarefa de Sistema no MINIX 3 (cont.)



(a)



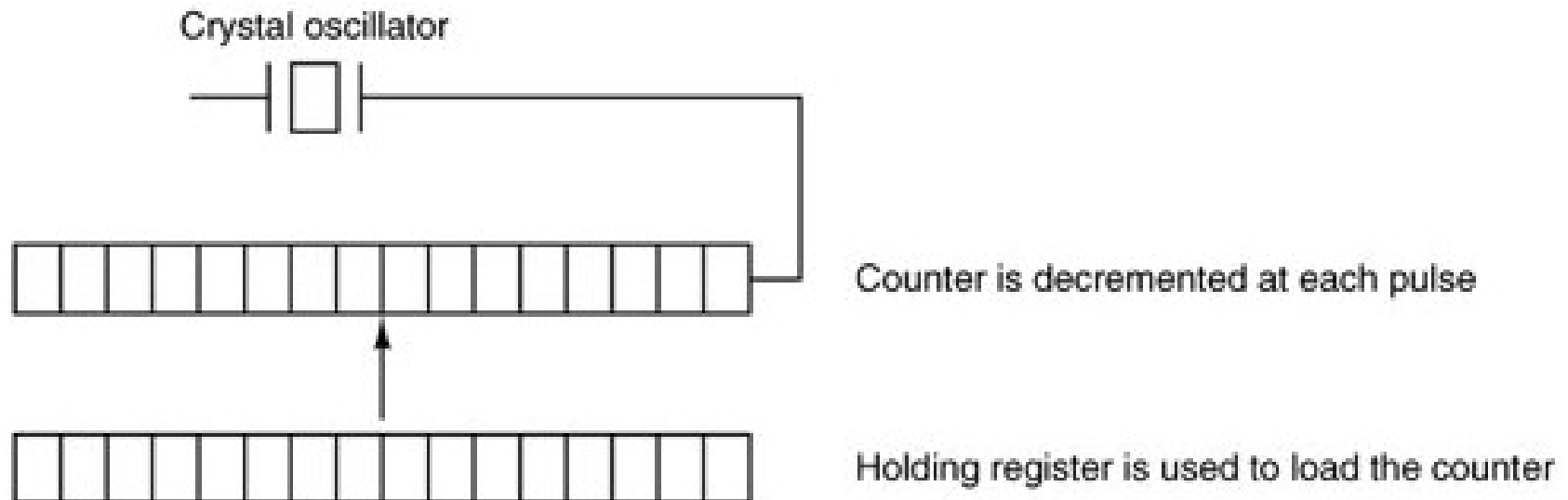
(b)

Tarefa de Clock no MINIX 3

- Clocks são essenciais para a operação de timesharing do sistema.
 - ex. mantém a hora do dia, evita monopolização de processador.
- No MINIX 3 a interface para o clock não é fornecida no diretório /dev/
- Clock task executa em espaço de kernel.
 - Não pode ser diretamente acessada por processos de usuário.
 - Só pode ser acessada via system task.

Tarefa de Clock no MINIX 3 (cont.)

- Hardware: oscilador de cristal, um contador e registrador de manutenção (holding register).
- Oscila entre 5-200 Mhz (dependendo do cristal).
- Sinal alimentado no contador. Quando chega a zero gera-se uma interrupção (holding register mantém o valor que será decrementado).



Tarefa de Clock no MINIX 3 (cont.)

- Software de clock
 - Hardware de clock gera interrupções em intervalos conhecidos.
 - Tarefas de software incluem:
 - 1) Manter a hora do dia
 - 2) Prevenir que processos executem mais do que o necessário.
 - 3) Contabilização (accounting) para o uso de CPU.
 - 4) Manipular a system call alarm feita por processos de usuário.
 - 5) Oferece watch dog timers para partes do sistema
 - 6) Profiling, monitoramento e reunião de estatísticas.

Tarefa de Clock no MINIX 3 (cont.)

Service	Access	Response	Clients
get_uptime	Function call	Ticks	Kernel or system task
set_timer	Function call	None	Kernel or system task
reset_timer	Function call	None	Kernel or system task
read_clock	Function call	Count	Kernel or system task
clock_stop	Function call	None	Kernel or system task
Synchronous alarm	System call	Notification	Server or driver, via system task
POSIX alarm	System call	Signal	User process, via PM
Time	System call	Message	Any process, via PM

Resumo dos serviços de clock.